

Hybrid Approach to Directed Fuzzing

Darya Parygina,
Timofey Mezhuev,
Daniil Kuts

June 26, 2025

ISP RAS, Sydr-Fuzz team

Automated Program Testing

Directed fuzzing:

- focuses on reaching predefined program location using some metrics
- used in patch testing, crash reproduction, static analysis report verification
- high **speed** of errors detection, **flexibility** in proximity metrics
- lacks the ability to overcome difficult program constraints

Symbolic execution:

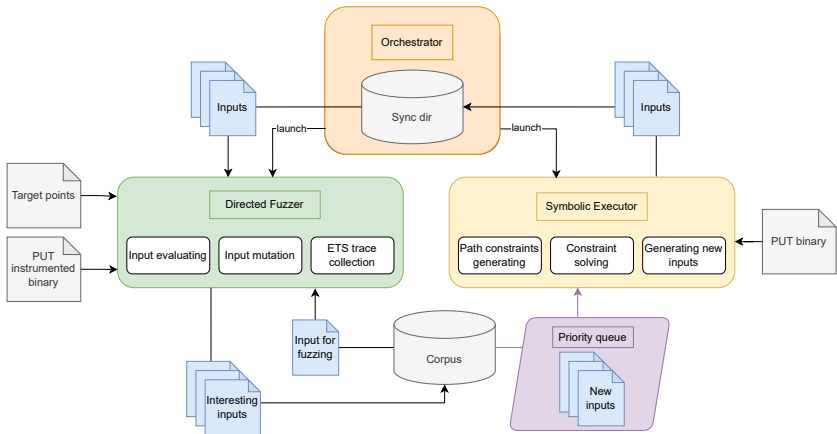
- interprets program instructions in terms of symbolic values
- discovers new program paths by inverting branch constraints
- **overcomes complicated constraints** with SMT-solvers assistance
- struggles from high overheads

Combination: high speed + flexibility + complex constraints handling

Existing Approaches to Directed Fuzzing

- **AFLGo**: weighted distance to target points as proximity metric, power scheduling based on simulated annealing algorithm
- **BEACON**: reachability analysis to prune irrelevant basic blocks
- **WAFLGo**: Critical Code Guidance for finding bugs in new commits
- **WindRanger**: Deviation Basic Blocks, taint tracking
- **FishFuzz**: adaptive dictionary creation and update strategy
- **Prospector**: precise call site identification with adaptive seed selection
- **Hybrid directed fuzzing**: symbolic execution assistance for discovering new coverage

Hybrid Directed Fuzzing with LibAFL-DiFuzz and Sydr



Seed Scheduling Algorithm

LibAFL-DiFuzz → Sydr:

- Fuzzer **corpus** directory → priority queue
- Input priority by interest score: ETS exploration + coverage growth
- Sydr takes inputs from the queue top

Sydr → LibAFL-DiFuzz:

- **Sync dir** – separate directory for generated inputs → corpus directory/objective directory
- LibAFL-DiFuzz performs synchronization from **Sync dir** at dynamically calculated time intervals: $\max(60, \text{import_time} * 3)$

Seed priority queue

- Schedules inputs for symbolic executor
- Based on binary heap
- New inputs are added in the sorted order
- Top element has the greatest priority

Scores for priority:

1. ETS score: flag for ETS exploration
2. coverage score: flag for new coverage
3. file score: file creation time / file size

Metadata contains:

- `is_interstring_ets` flag: whether input is interesting for ETSFeedback
- `is_interstring_map` flag: whether input is interesting for MapFeedback
- ETS trace: list of IDs of visited basic blocks belonging to at least one ETS
- reached target points: list of `<source file name, line number>` pairs
- `is_crash` flag: input produces target crash
- `is_timeout` flag: target hangs on this input

Objective sorting and minimization

Objective: crash/hang/target point reaching

Minimization: unique ETS trace (sequence of basic blocks that are dominators of the target basic block)

Sorting: reached target points

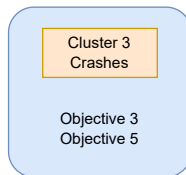
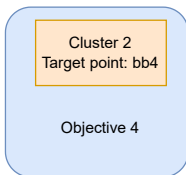
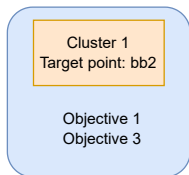
Objective 1
ETS trace: [bb1, bb2 , bb5]

Objective 2
ETS trace: [bb1, bb2 , bb5]

Objective 3
ETS trace: [bb1, bb2 , bb3, bb5], CRASH

Objective 4
ETS trace: [bb1, bb4 , bb5]

Objective 5
ETS trace: [], CRASH



Sydr-Fuzz pipeline

Configuration:

```
[sydr]
args = "-j 4"
target = "/target_sydr @@"
jobs = 3

[difuzz]
target = "/target_libafl_difuzz @@"
args = "-j2 -i /corpus -e /ets.toml"
path = "/fuzzer_libafl_difuzz"
```

Pipeline:

- Launch LibAFL-DiFuzz
- Launch Sydr workers, take inputs from priority queue, put inputs to sync dir
- Parse fuzzer logs, print statistics
- Update priority queue every minute
- At stop conditions: kill all workers, minimize and sort objectives

Target points for experiments

Project	TP ID	Location	TimeOut
cxxfilt	<i>cxxfilt₁</i>	libiberty/cp-demangle.c: 1596	45 min
	<i>cxxfilt₂</i>	libiberty/cplus-dem.c: 4319	
giflib	<i>giflib</i>	util/gifsponge.c: 61	30 min
jasper	<i>jasper</i>	src/libjasper/ mif/mif_cod.c:491	30 min
libming	<i>libming</i>	util/outputsript.c:1493	150 min
libxml	<i>libxml₁</i>	valid.c:1181	60 min
	<i>libxml₂</i>	valid.c:1189	

Experiments: TTE

TTE (Time to Exposure) – time spent to reach target point

TP ID	Sydr-Fuzz	LibAFL-DiFuzz	AFLGo	BEACON	WAFLGo
<i>cxxfilt₁</i>	22	175	49	60	41
<i>cxxfilt₂</i>	136	-	63	55	3043
<i>giflib</i>	3	3	18	4	5
<i>jasper</i>	18	16	16	50	-
<i>libming</i>	119	127	412	53	-
<i>libxml₁</i>	8	14	28	43	107
<i>libxml₂</i>	61	300	343	44	-

TP ID	Sydr-Fuzz	LibAFL-DiFuzz	WindRanger	FishFuzz	Prospector
<i>cxxfilt₁</i>	22	175	44	384	470
<i>cxxfilt₂</i>	136	-	704	1208	347
<i>giflib</i>	3	3	55	41	45
<i>jasper</i>	18	16	81	287	22
<i>libming</i>	119	127	-	862	59
<i>libxml₁</i>	8	14	666	2285	1694
<i>libxml₂</i>	61	300	1108	-	-

Conclusion

Implemented hybrid approach to directed fuzzing with LibAFL-DiFuzz as directed fuzzer and Sydr as dynamic symbolic executor.

- Novel seed scheduling algorithm based on target-related interestingness and coverage
- Minimization and sorting of objective seeds according to the target-related information
- Implemented approach in Sydr-Fuzz pipeline
- Evaluated and compared with pure LibAFL-DiFuzz, AFLGo, BEACON, WAFLGo, WindRanger, FishFuzz, and Prospector
- Got 3 out of 7 best results with speedup up to 1.86 times from the second best results
- Got significant improvements on 3 out of 7 examples comparing with pure LibAFL-DiFuzz, reached an undiscovered target point

Thank you! Questions?